

BÁO CÁO KỸ THUẬT MINI-PROJECT 1

XÂY DỰNG ỨNG DỤNG PROGRESSIVE WEB APP KHẢO SÁT ĂM THỰC VKU (VKU FOOD SURVEY PWA)

Môn học: Phát triển Ứng dụng Di động Đa nền tảng

Đề tài: Progressive Web App (Week 3 - Mini-Project 1)

Tên miền Demo (HTTPS): <https://chidung7271.id.vn>

Giảng viên hướng dẫn: TS. Nguyễn Thanh Tuấn

Sinh viên thực hiện: Sinh viên Khoa KHMT - VKU

Mã nguồn: GitHub Public Repository

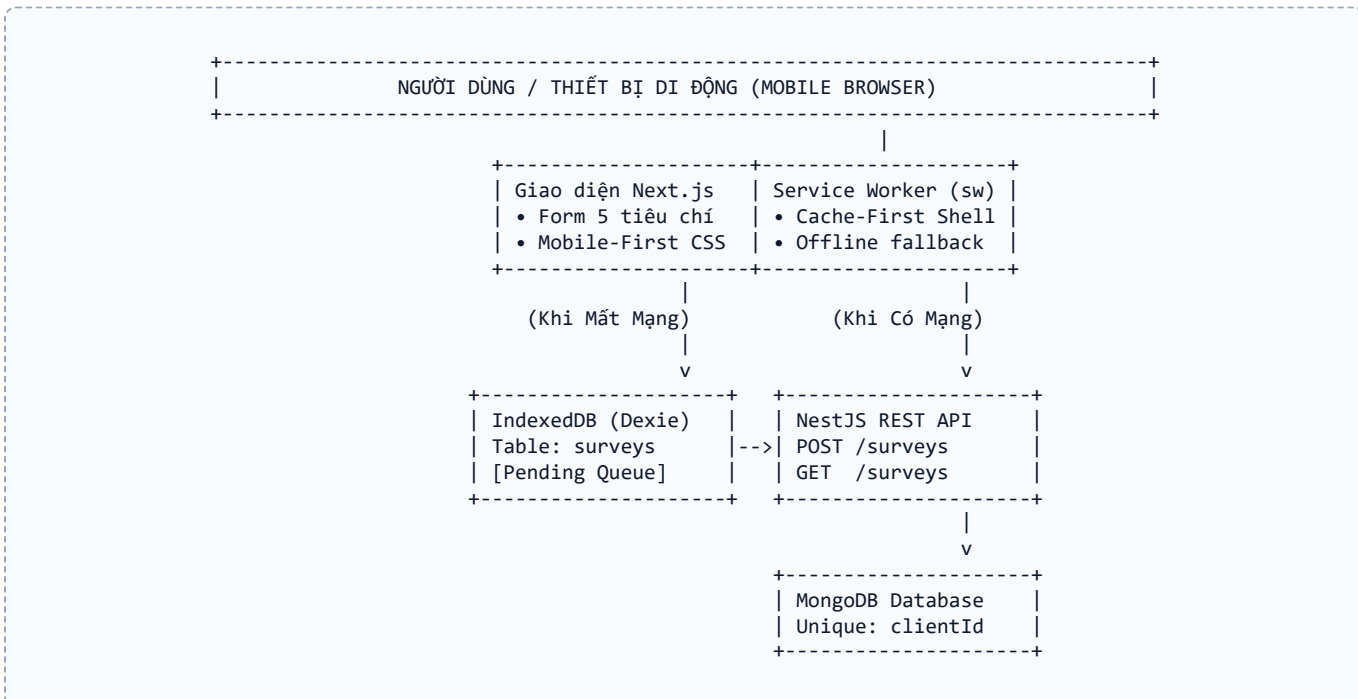
1. GIỚI THIỆU & MỤC TIÊU ĐỀ TÀI

Trong khuôn khổ môn học *Phát triển Ứng dụng Di động Đa nền tảng* tại VKU, Mini-Project 1 hướng tới xây dựng giải pháp khảo sát chất lượng dịch vụ ăn uống và căng tin sinh viên bằng công nghệ **Progressive Web App (PWA)**. Ứng dụng giải quyết triệt để sự cố mạng chậm chạp tại các khu vực căng tin đông người bằng triết lý **Offline-First**.

Dự án đáp ứng đầy đủ 5 tiêu chí cốt lõi của một ứng dụng PWA chuẩn:

- Installable:** Cài đặt độc lập lên màn hình chính thiết bị di động (Standalone mode) qua Web App Manifest, hiển thị biểu tượng và splash screen riêng biệt.
- Offline-First:** Vận hành tin cậy ngay cả khi mất mạng nhờ Service Worker (Cache API) và IndexedDB (Dexie.js).
- Fast & Responsive:** Thời gian nạp App Shell dưới 1 giây từ local cache; giao diện chuẩn Mobile-first tối ưu cho màn hình cảm ứng di động.
- Auto-Sync & Deduplication:** Tự động đồng bộ hóa dữ liệu lên máy chủ ngay khi có mạng trở lại mà không tạo dữ liệu trùng lặp (Idempotency với UUID clientId).
- Secure:** Triển khai bắt buộc trên giao thức bảo mật HTTPS (Vercel / Cloudflare Pages).

2. KIẾN TRÚC TỔNG THỂ & CÔNG NGHỆ LỰA CHỌN



Thành phần	Công nghệ sử dụng	Vai trò trong hệ thống
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS	UI Mobile-first, Render App Shell, Star rating tương tác

Thành phần	Công nghệ sử dụng	Vai trò trong hệ thống
Local Storage	IndexedDB (Dexie.js Wrapper)	Lưu trữ bản nháp khảo sát offline dạng bảng JSON
PWA Core	Service Worker, Cache API, Manifest	Pre-cache App Shell, đón bắt HTTP request, background sync
Backend API	NestJS 10, TypeScript, Mongoose	REST API /surveys, validate DTO, cơ chế chống duplicate
Database	MongoDB (Atlas Cloud / Local)	Lưu trữ dữ liệu khảo sát chính thức với unique index clientId

3. HIỆN THỰC CƠ CHẾ OFFLINE-FIRST & SERVICE WORKER

3.1. Cấu hình Web App Manifest

Tập `public/manifest.json` khai báo định danh ứng dụng với `display: "standalone"` giúp ẩn thanh URL trình duyệt, màu chủ đạo `theme_color: "#1e40af"` (Xanh VKU), cùng hệ thống icon đa độ phân giải (192x192, 512x512 maskable). Người dùng có thể nhấn nút "Cài App" trực tiếp trên giao diện để đưa icon ra màn hình chính.

3.2. Vòng đời Service Worker & Chiến lược Caching

Service Worker (`public/sw.js`) triển khai 3 pha vòng đời cốt lõi:

- **Install Event:** Mở cache `vku-food-survey-v1` và pre-cache App Shell tĩnh gồm các trang HTML (`/`, `/survey`, `/history`), CSS, icon. Sử dụng `skipWaiting()` để kích hoạt phiên bản mới ngay lập tức.
- **Activate Event:** Quét và xóa các phiên bản cache lỗi thời; gọi `clients.claim()` để kiểm soát toàn bộ tab đang mở.
- **Fetch Event:** Triển khai chiến lược caching phân tách theo loại tài nguyên:
 - **App Shell & Assets tĩnh (`/_next/`, `icons`, `css`):** Áp dụng *Cache-First*. Ưu tiên đọc từ Cache để nạp tức thì dưới 1 giây; nếu chưa có mới tải từ mạng.
 - **Dữ liệu danh sách GET (`/surveys`):** Áp dụng *Network-First*. Lấy dữ liệu mới nhất; nếu offline sẽ lấy bản snapshot từ cache.
 - **Gửi khảo sát POST (`/surveys`):** Áp dụng *Network-Only* (không cache HTTP POST bằng Cache API). Tăng ứng dụng lưu trữ offline vào IndexedDB.
 - **Kiểm tra kết nối (`/api/ping`):** Áp dụng *Network-Only* để phát hiện trạng thái offline tức thời.

3.3. Lưu trữ Cấu trúc Offline với IndexedDB (Dexie.js)

Cơ sở dữ liệu cục bộ `surveyDB` được cấu hình qua Dexie với bảng `surveys` có cấu trúc schema: `'++id, clientId, status, createdAt'`. Mỗi bản ghi khảo sát cục bộ có cấu trúc:

```
{
  id: 1, // Khóa chính tự tăng cục bộ
  clientId: "550e8400-e29b-41d4", // UUID định danh duy nhất chống duplicate
  status: "pending" | "synced", // Trạng thái đồng bộ (pending, synced, failed)
  data: {
    foodName: "Cơm tấm sườn", location: "Cảng tin Khu A", category: "Cơm",
    tasteRating: 5, priceRating: 4, hygieneRating: 5, qualityRating: 5, overallRating: 5,
    price: 25000, comment: "Rất ngon và vệ sinh"
  },
  createdAt: "2026-09-03T08:30:00.000Z"
}
```

4. CƠ CHẾ TỰ ĐỘNG ĐỒNG BỘ (AUTO-SYNC) & CHỐNG TRÙNG LẬP (DEDUPLICATION)

4.1. Quy trình phát hiện kết nối & Đồng bộ tự động

1. Khi bấm "Gửi khảo sát" lúc mất mạng: Dữ liệu kèm UUID `clientId` được lưu vào IndexedDB với status: `'pending'`. Màn hình thông báo ngay: *"Đã lưu khảo sát trên thiết bị. Khảo sát sẽ được đồng bộ khi có mạng."*
2. Hệ thống kích hoạt 2 kênh giám sát mạng:
 - Kênh 1: Lắng nghe sự kiện `window.addEventListener('online', ...)` và probe heartbeat `/api/ping` mỗi 2.5s.
 - Kênh 2: Đăng ký Background Sync API (tag `'vku-food-sync'`).
3. Khi mạng phục hồi: Hàm `syncPendingSurveys()` quét toàn bộ bản ghi `pending`, gửi tuần tự qua POST `/surveys`.
4. Khi server phản hồi HTTP 201, hàm `markAsSynced(clientId)` đổi trạng thái sang `synced`, badge đổi sang  **Đã đồng bộ** và thông báo toast cho người dùng.

4.2. Kỹ thuật Chống Trùng lập (Idempotency với UUID clientId)

Trong môi trường mạng không ổn định, request sync có thể bị gửi lặp lại:

- Ở Frontend: Mỗi khảo sát tạo duy nhất 1 mã UUID `clientId` (qua `crypto.randomUUID()`) ngay lúc tạo.

- Ở Backend: Schema Mongoose thiết lập `clientId`: { type: String, unique: true, index: true }. Service kiểm tra `findOne({ clientId })`. Nếu đã tồn tại, backend trả về kết quả thành công và đánh dấu `isDuplicate: true` mà không tạo thêm bản ghi thừa.

5. ĐÁNH GIÁ KẾT QUẢ KIỂM THỬ

Dự án đã trải qua quá trình kiểm thử thực tế theo các tiêu chuẩn kỹ thuật nghiêm ngặt:

Kịch bản kiểm thử	Hành vi hệ thống	Kết quả thực tế
1. Cài đặt PWA	Trình duyệt hiện nút cài đặt, tạo app standalone	ĐẠT (PASS) Mở full màn hình
2. Nạp App khi Offline	DevTools Network = Offline, tải lại trang (F5)	ĐẠT (PASS) App Shell nạp tức thì từ Cache
3. Nhận diện mạng	Chuyển đổi Network Offline / No throttling	ĐẠT (PASS) Badge đổi ● Offline / ● Online
4. Gửi khảo sát Offline	Điền form và nhấn Gửi khi mất mạng	ĐẠT (PASS) Lưu an toàn vào IndexedDB (pending)
5. Tự động đồng bộ	Bật lại Internet (Network = Online)	ĐẠT (PASS) Tự đẩy lên NestJS & MongoDB
6. Kiểm tra trùng lặp	Bấm nút đồng bộ lại nhiều lần	ĐẠT (PASS) MongoDB chỉ lưu duy nhất 1 bản ghi

6. TRIỂN KHAI HỆ THỐNG (DEPLOYMENT)

- **Frontend PWA:** Triển khai trên **Vercel**, tự động cấp phát chứng chỉ HTTPS miễn phí. Tích hợp tên miền cá nhân `https://chidung7271.id.vn` (hoặc subdomain `vku-food.chidung7271.id.vn`) thông qua bản ghi DNS CNAME trở về `cname.vercel-dns.com`.
- **Backend API:** Triển khai trên **Render.com** / Railway, chạy NestJS production build.
- **Cơ sở dữ liệu:** Cụm đám mây **MongoDB Atlas** (M0 Free Tier) tại Singapore cho độ trễ kết nối cực thấp.

7. KẾT LUẬN & HƯỚNG PHÁT TRIỂN TUẦN TIẾP THEO

Kết luận: Mini-Project 1 đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra cho bài tập Progressive Web App: xây dựng thành công ứng dụng khảo sát ẩm thực VKU chuẩn Mobile-first, cài đặt độc lập, nạp offline qua Service Worker, lưu trữ bền vững với IndexedDB Dexie, và tự động đồng bộ hóa thông minh chống trùng lặp.

Kế hoạch tiếp theo (Theo nội dung Slide 11 - Week 4):

- Đóng gói mã nguồn PWA này thành tệp cài đặt **Android APK nguyên bản** sử dụng **Capacitor Bridge** (`@capacitor/core`, `@capacitor/android`).
- Tích hợp phần cứng thiết bị di động (Camera chụp ảnh món ăn căng tin, Geolocation định vị vị trí căng tin).

Giảng viên hướng dẫn

Sinh viên thực hiện

TS. Nguyễn Thanh Tuấn

Sinh viên Khoa KHMT - VKU